

Unidad II: De la abstracción a la implementación

Clase 3 – Cola con prioridad y Conjunto: elegir estructura según el problema

Objetivo

Comprender que no todas las estructuras sirven para todos los problemas.

Tema

- TDA Cola con Prioridad.
- TDA Conjunto.
- Estrategias de implementación.
- Introducción al Lenguaje Java
- Implementación de TDAs

Introducción a Lenguaje Java

***Cuadrado Estrebou, María Fernanda Trutner, Guillermo Hernán. Revisión: J.Tempio.
Programación II. Apuntes de cátedra.*

En esta unidad se introducirán los conceptos del lenguaje Java que se van a necesitar a lo largo del curso. No es éste un curso para aprender a programar en este lenguaje sino que es un curso en que se lo va a utilizar como herramienta para ejercitar todos los conceptos vistos.

Uno de los aspectos importantes a tener en cuenta antes de comenzar, es que el lenguaje es sensible a mayúsculas y minúsculas, es decir se deben utilizar las variables y los métodos tal cual fueron definidos.

Tipos de datos

Los tipos de datos básicos en Java que se van a utilizar a lo largo del curso, son los siguientes:

- boolean: los valores posibles que pueden tomar las variables de este tipo son true o false únicamente
- int: valores enteros que tienen un tamaño de 32 bits
- short: valores enteros que tienen un tamaño de 16 bits
- char: representa a un carácter, y tiene un tamaño de 16 bits

- byte: representa a 8 bits
- float: es un valor numérico con decimales con un tamaño de 32 bits
- long: es un valor numérico con decimales con un tamaño de 64 bits
- double: es un valor numérico con decimales con un tamaño de 64 bits

Variables

Otro aspecto para considerar es cómo se lleva a cabo la declaración de variables en Java, y las mismas pueden ser de la siguiente manera:

- declaración de una variable sola:
`<tipo> <nombre_variable> ;` por
 ejemplo: `int cantidad;`
- declaración de más de una variable del mismo tipo:
`<tipo> <nombre_variable_1>,<nombre_variable_2>,<nombre_variable_3>;` por ejemplo:
`int cantidad, contador, resultado;`
- declaración y asignación de valor a una variable:
`<tipo> <nombre_variable> = <valor>;` por
 ejemplo: `int contador = 0;`
 o para el caso de variables del tipo char se debe usar para el valor entre “”: `char c = 'f';`

Como se vio en algunos de los ejemplos anteriores para la asignación de valores a una variable se utiliza el operador = (por ejemplo: `variable = 5`).

El alcance de la variable está dado por el bloque en el que se declara. Una variable es visible dentro del bloque en el que fue declarada y dentro de todos los sub-bloques que se definan dentro de éste.

Operadores

Aritméticos

Las operaciones aritméticas básicas sobre números son las siguientes:

- Para indicar que un número es negativo se usa el símbolo - (por ejemplo -5). Este mismo símbolo es utilizado también para la operación de resta (por ejemplo `x = x - 8`);
- +: operador de suma.
- *: operador de multiplicación.
- /: operador de división.

- %: operador de resto ó módulo, es decir, devuelve el resto de una división (por ejemplo, $x \% 2$; el resultado de esta operación es 1 si el número es impar y 0 si el número par).
- ++: operador de incremento en uno (por ejemplo si tenemos `int x = 2; x++;`, entonces el valor que toma x es 3).
- --: operador de decremento en uno (por ejemplo si tenemos `int x = 2; x--;`, entonces el valor que toma x es 1).

Relacionales

Las operaciones de comparación básicas sobre números son las siguientes:

- <: operador menor, es decir, si tenemos $x < z$, la comparación devolverá true si x tiene un valor menor que el de z, y false en caso contrario.
- >: operador mayor, es decir, si tenemos $x > z$, la comparación devolverá true si x tiene un valor mayor que el de z, y false en caso contrario.
- <=: operador menor o igual, es decir, si tenemos $x \leq z$, la comparación devolverá true si x tiene un valor menor o igual que el de z, y false en caso contrario.
- >=: operador mayor o igual, es decir, si tenemos $x \geq z$, la comparación devolverá true si x tiene un valor mayor o igual que el de z, y false en caso contrario.
- ==¹: operador de igualdad, es decir, si tenemos $x == z$, la comparación devolverá true si x tiene un valor igual que el de z, y false en caso contrario.
- !=: operador distinto, es decir, si tenemos $x != z$, la comparación devolverá true si x tiene un valor distinto que el de z, y false en caso contrario.

Lógicos

Además, de las operaciones sobre números están las operaciones sobre valores lógicos:

- !: operador de negación, es decir, si tenemos `!valor`², la comparación devolverá true si el valor que tiene valores falso, y false en caso contrario.
- |: operador O lógico. • &&: operador Y lógico.

Estructuras de control

Dentro de las instrucciones básicas veremos la notación para secuencia, el condicional, condicional múltiple y algunos operadores de ciclos.

¹ Tener especial cuidado de no omitir un signo =, ya que el lenguaje lo tomaría como una asignación y no daría error, pero cambiaría el significado o comportamiento de la operación

² valor debe ser una expresión de tipo booleano

Bloques

En Java, los bloques de sentencias se delimitan con llaves ({ y }). Se comienza un bloque con una llave de apertura y se finaliza con una llave de cierre. En muchos de los casos, si el bloque contiene una única sentencia, se pueden omitir las llaves, aunque para evitar errores y malos entendidos, se recomienda utilizarlas siempre. La indentación se utilizará a modo simbólico para facilitar la lectura del código, sin embargo, el compilador lo ignora totalmente.

Secuencia

Para la separación de secuencias de instrucciones se utiliza ;.

Condicional Simple

El condicional simple se utiliza para validar una condición, y según sea verdadera o falsa se ejecuta un bloque de sentencias.

```
if (<condicion_booleana >){
    instruccion 1;
    instruccion 2;
    instruccion 3;
    ---
} else {
    instruccion 4;
    instruccion 5;
    instruccion 6;
    ---
}
```

El else en el condicional es opcional. En caso de no utilizar las llaves para armar un bloque de instrucciones, la primera instrucción será la que forme parte del if ó else y las siguientes quedan fuera del condicional.

Condicional múltiple

Para las evaluaciones de diferentes condiciones para un mismo valor se puede utilizar el switch. En Java el switch puede evaluar int , char , String , enum, byte, short

Ejemplo con String:

```
String opcion = "SI";
```

```
switch(opcion) {
    case "SI":
        System.out.println("Aceptado");
        break;
}
```

Lo que **no puede evaluar**:

- double
- float
- boolean

Switch evalúa tipos discretos (int, char, String, enum), no valores continuos.

```
switch (<expresión_entera>){
    case valor1:
        instrucción 1;
        instrucción 2;
        ...
        break;
    case valor2:
        instrucción 3;
        instrucción 4;
        ...
        break;
    case valor3:
        instrucción 5;
        instrucción 6;
        ...
        break;
    default :
        instrucción 7;
        instrucción 8;
        ...
}
```

En cada case es aconsejable colocar un break, debido a que si no se coloca un break el lenguaje sigue evaluando las condiciones de los siguientes case, y en caso de que otro sea verdadero también lo ejecuta. Además, si la sentencia switch tiene el default y no se coloca un break en cada case, luego de ejecutar el case correspondiente siempre va a terminar ejecutando lo que esta definido dentro del default.

Ciclos

Para la ejecución de ciclos vamos a ver tres instrucciones, while, for y do..while. Las dos primeras se pueden ejecutar como mínimo cero veces, a diferencia del do..while que se ejecuta como mínimo una vez, debido a que primero hace la ejecución y luego evalúa la condición.

```
while (<condicion_booleana >){
    expresion 1;
    expresion 2;
    ...
}
```

Si luego de la condición del while no se colocan las llaves, el lenguaje va a considerar como expresión de ejecución solamente la primera, y no las subsiguientes (este mismo comportamiento es válido para la sentencia for que describiremos a continuación).

Otra opción para un ciclo es el for:

```
for (<inicializacion_variable> ; <condición_booleana> ; <
    variacion_variable> ){
    expresion 1;
    expresion 2;
    ...
}
```

La variable que se inicializa y utiliza dentro del for puede ser declarada previo al for o dentro del mismo (como en el ejemplo que sigue). De acuerdo a dónde se declare será el alcance que tenga, es decir, si se declara en el mismo for sólo se podrá utilizar en las instrucciones dentro del ciclo del for. La variación de la variable puede ser un incremento o un decremento. Un ejemplo de uso del for podría ser para sumar los números del 1 al 10, con lo cual se haría:

```
int suma = 0;
for (int i = 1; i <= 10 ; i++){
    suma = suma + i;
}
```

En este caso tenemos declarada la variable i del for en el mismo, ya que no se requiere fuera del él.

Ahora supongamos que estamos buscando un valor en un arreglo de enteros de 20 elementos y queremos conocer no sólo si existe el valor, sino en qué posición del mismo, en cuyo caso haríamos lo siguiente:

```
int buscarValor(int[] a,int valor, int N) {
    int i;
    for(i=N-1; i > -1 && a[i]!=valor ;i--)
        ;
    return i;
}
```

Por último, nos queda presentar el do..while:

```
do {
    instruccion 1;
    instruccion 2;
    ...
} while (<condicion_booleana>)
```

El do..while, a diferencia del las otros dos sentencias de ciclos, lo que hace primero es ejecutar las instrucciones y luego evaluá la condición, con lo cual, al menos una vez se ejecutan todas las instrucciones independientemente del valor de la condición.

Arreglos

Luego de haber descripto las diferentes sentencias de ciclos vamos a ver como se declaran y usan los arreglos en Java.

Declaración

La forma de declarar un arreglo es la siguiente:

`<tipo_datos_arreglos>[] <nombre_arreglo>;` por ejemplo, si queremos declarar un arreglo de enteros haríamos lo siguiente:

```
int[] valores;
```

Dimensionamiento

Los arreglos, por tratarse de una estructura de tamaño fijo, antes de poder utilizarlos se le debe asignar la cantidad de elementos que va a contener, es decir, dimensionarlo:

```
<nombre_arreglo> = new <tipo_datos_arreglos>[<dimension>]
```

La palabra reservada `new`, se utilizará en el lenguaje, cada vez que se quiera reservar memoria para una variable que no sea simple. Para los enteros, reales y caracteres, como se ha visto, no es necesario hacer la reserva de memoria, ya que la declaración de la variable lleva inmersa la reserva de memoria.

Por ejemplo, si al arreglo `valores` utilizado para ejemplificar en la parte de declaración, le queremos decir que va a contener 100 elementos, entonces lo que haríamos es: `valores = new int[100];`

Otra forma de dimensionar un arreglo, y a su vez inicializarlo con valores es la siguiente: `int[] valores = {5,8,15,20};`

en este caso el arreglo toma una dimensión 4, con los valores 5, 8, 15 y 20 en las posiciones 0, 1, 2 y 3 respectivamente.

Utilización

Debemos tener en cuenta que los arreglos en Java tienen su primer elemento en la posición cero, es decir, si declaramos un arreglo de dimensión 100, lo vamos a poder recorrer desde 0 a 99. A continuación se ve un ejemplo de recorrido del arreglo `valores` para inicializarlo con valores:

```
for (int i = 0; i < 100;i++){
    Valores[i] = i + 2;
}
```

Paquetes y salida

El lenguaje provee conjuntos de librerías que pueden ser utilizadas dentro de nuestros programas. En Java las librerías se llaman paquetes, y se agrupan en sub-paquetes. Por ejemplo, hay un paquete `net` y dentro del mismo hay un paquete `URL`. Para hacer referencia entonces a la `URL`, se utilizará la notación de punto, por ejemplo `util.URL` y si queremos

utilizar una función de la librería, hay que invocar un método dentro de la librería, por ejemplo para recuperar el contenido de una URL se podría hacer así: `util.URL.getContents()`

Un paquete particular que vamos a utilizar es el que sirve para imprimir por pantalla. Es el paquete `System.out` y el método se llama `println`, entonces para imprimir algo, estos son ejemplos:

```
int n = 3;
char c = 'f';
System.out.println(n);
System.out.println(c);
System.out.println("Hola!");
```

Métodos

Por último, vamos a ver dos conceptos particulares de los lenguajes orientados a objetos como es el caso de Java, que son los métodos y clases. El concepto de métodos y clases que daremos es básico y no tiene por objetivo dar un conocimiento del paradigma de Orientación a Objetos, sino dar las herramientas básicas para poder trabajar a lo largo del curso.

En Java todas las operaciones se llevan a cabo a través de métodos (se puede hacer una similitud a funciones y procedimientos de los lenguajes estructurados) los cuales se declaran de la siguiente manera:

```
<tipo_retorno_método><nombre_método>(<tipo_parametro>< parametro1>
..., <tipo_parametro> <parámetro_n>){
    instruccion1;
    instruccion2;
    ...
}
```

En el caso de que el método no devuelva valor se lo declara del tipo `void`. Por otra parte, el método puede no contener parámetros, en cuyo caso es igualmente obligatorio colocar los paréntesis sin ningún parámetro, es decir, se colocaría:

```
<tipo_retorno_método><nombre_método>(){
    instruccion1;
    instruccion2;
    ...
}
```

A continuación, daremos dos ejemplos de métodos, el primero devolverá un valor que es la suma de dos variables que se dan como parámetro, y el segundo será un método que no devuelve valor sino que imprime por pantalla los valores de un arreglo dado.

Ejemplo 1: Suma de dos valores dados

```
int sumar(int x, int y){
    int sum = 0;
    sum = x+y;
    return sum;
}
```

Esto se puede escribir de la siguiente manera que da el mismo resultado, es decir, son formas equivalentes de decir lo mismo:

```
int sumar(int x, int y){
    return (x+y);
}
```

Ejemplo 2: impresión por pantalla de los elementos de un arreglo de dimensión

10.

```
void imprimirArreglo( int [] a) {
    for (int i = 0; i < 10;i++) {
        System.out.println (a[i]);
    }
}
```

Clases

Luego de haber hablado de los métodos, nos resta describir el concepto de clases. En los lenguajes Orientados a Objetos siempre vamos a estructurar nuestros programas o sistemas mediante clases. Las clases las podríamos comparar a las estructuras de los lenguajes estructurados, aunque la definición correcta de las clases es que son la definición de un objeto, en donde un objeto representa a alguna entidad de la vida real (por ejemplo, un objeto puede ser una persona, una casa, un auto, una formula, etc.). A continuación, veremos cómo es la declaración y el uso de una clase.

Declaración

Vamos a declarar una clase con sus componentes, esos componentes son variables de la clase y sus métodos. Para indicar que es una clase se utiliza la palabra reservada del lenguaje class.

```
class <nombre_clase>{
    <tipo_variable> <nombre_variables>; //variables de la clase
    <tipo_metodo> <nombre_metodo> (<parámetros_metodo>){
        instrucción 1;
        instrucción 2;
        ...
    }
}
```

Como ejemplo, vamos a declarar una clase punto, en donde tenemos dos componentes que representan las coordenadas x e y del punto:

```
class Punto{
    int x;
    int y;
}
```

Uso de clases como variables

El uso de las clases es similar al uso de los tipos primitivos, es decir, se debe declarar una variable del tipo de la clase:

```
<nombre_clase> <nombre_variable>;
```

Por ejemplo, si queremos declarar una variable p del tipo Punto debemos hacer:

```
Punto p;
```

Y luego antes de poder usar cualquier variable cuyo tipo sea una clase (es decir, no sea una variable de tipo primitivo) se le debe asignar memoria de la siguiente manera:

```
<nombre_variable> = new <nombre_clase>();
```

Continuando con el ejemplo de la variable p del tipo Punto, para asignar memoria haríamos lo siguiente:

```
p = new Punto();
```

Cabe aclarar que la declaración de una variable y su asignación de memoria se puede hacer en una misma instrucción de la siguiente manera:

```
<class_name> <nombre_variable> = new <nombre_clase>();
```

Utilizando lo mencionado anteriormente en nuestro ejemplo nos quedaría:

```
Punto p = new Punto();
```

Por último, nos resta ver como podemos acceder a los componentes de las clases, es decir, las variables³ y métodos. Para ello se utiliza el punto "." de la siguiente manera:

```
<nombre_variable>.<nombre_componente>
```

Es decir, si queremos asignarles valores a las coordenadas x e y del punto p lo que haríamos es lo siguiente:

³ Debemos tener en cuenta que no hemos introducido el concepto de métodos privados y públicos hasta el momento, ya que sólo podríamos acceder desde afuera si el método es público. Este concepto se va a ver en los próximos párrafos. No se verá en el curso el concepto de variables o métodos protegidos

p.x = 5; p.y = 10;

En lo referido a métodos y variables que hemos venido trabajando, existe el concepto de alcance o visibilidad de los métodos y variables, y es lo que nos permite indicar si un método o variable podrá ser utilizado por quién usa esa clase o no. Es decir, si un método o variables es privado solo se podrá utilizar dentro de la clase, en cambio si es público podrá ser utilizado tanto dentro de la clase como por quién usa esa clase. Para indicar que un método o variable es público se utiliza la palabra reservada del lenguaje public, y para indicar si es privado la palabra private (por defecto si no se indica nada el lenguaje considera que la variable o método es privada). Supongamos el siguiente ejemplo para mostrar lo mencionado anteriormente:

```
public class Persona {
    String nombre;
    String apellido;
    public String getNombre() {
        return nombre;
    }
    public void setNombre(String nombre) {
        this.nombre = nombre;
    }
    public String getApellido() {
        return apellido;
    }
    public void setApellido(String apellido) {
        this.apellido = apellido;
    }
    public String getCalle() {
        return calle;
    }
    public void setCalle(String calle) {
        this.calle = calle;
    }
}
```

Tenemos una clase que representa una persona, que tiene un nombre, apellido, la calle, número y ciudad en donde vive. Para asignarle el nombre y el apellido a la persona tenemos un método público y para asignarle la calle, número y ciudad otro. Estos dos métodos serán públicos y se deberán ser utilizados por quién usa la clase, pero las variables en donde guardamos esos valores no podrán ser accedidas por quién usa la clase con lo cual serán privadas. Para el ejemplo se utilizará un tipo de dato provisto por Java que es String que simboliza una cadena de caracteres y se puede imprimir como vimos anteriormente.

TDA.Conceptos y especificación

*****Cuadrado Estrebou, María Fernanda Trutner, Guillermo Hernán. Revisión: J.Tempio.
Programación II. Apuntes de cátedra.***

En este capítulo vamos a introducir el concepto de TDA, qué son, cómo se definen y cómo se utilizan. Vamos a introducir también la notación en lenguaje de programación que vamos a utilizar. Para los ejemplos, vamos a utilizar los TDA Pila, Cola, Cola con Prioridad, Conjunto y Diccionario.

Tipos de Datos Abstractos: Definición

Ya hemos definido en cursos anteriores los conceptos de Tipo de Dato y de Estructura de Dato. Vamos a recordar sus definiciones para poder introducir el nuevo concepto, que será el eje de la materia.

Cuando hablamos del tipo de dato de una variable, nos referimos al conjunto de valores que puede tomar una variable, y el conjunto de operaciones que se pueden aplicar sobre el mismo. Por ejemplo, si tomamos el tipo de dato int sabemos que el conjunto de valores que puede tomar son todos los enteros y el conjunto de operaciones serán la suma, resta, multiplicación, división y todas las demás que permita el lenguaje utilizado. Se los puede clasificar según su contenido en simples o estructurados, según contenga un único valor o un conjunto de valores. Ejemplos de tipos simples son enteros, reales, caracteres, etc. Ejemplos de estructurados pueden ser arreglos o las estructuras de C. Si son estructurados, se pueden clasificar en homogéneos o heterogéneos según sus elementos sean de un mismo tipo (por ejemplo arreglos) o de diferentes tipos (por ejemplo estructuras). También podríamos clasificarlos en predefinidos por el lenguaje o definidos por el usuario.

Una estructura de datos es una definición que se realiza utilizando los mecanismos propuestos por el lenguaje, que permite almacenar un conjunto de valores. Según la estructura definida será la forma de accederlas.

Un tipo de dato abstracto (TDA) es un tipo de dato según lo explicado más arriba, donde el concepto de abstracción está asociado al modo en que se definen los TDA. Este modelo estará formado por las operaciones que tiene y el comportamiento

asociado a esas operaciones, es decir, estará centrado en qué hace el TDA y no cómo lo hace, es decir, la abstracción significa el ocultamiento del cómo.

Especificación de un TDA

Como dijimos en la definición de TDA, vamos a separar qué hace un TDA de cómo lo hace. Y para entender esta diferencia, veremos cómo se pueden definir TDA y utilizarlos sin necesidad de conocer cómo se hace cada una de las operaciones. De hecho, para una

misma definición de TDA puede haber diferentes maneras de realizar lo definido por ese TDA.

A la definición del TDA la llamaremos especificación y a la forma de llevar a cabo lo definido por un TDA lo denominaremos implementación. Por lo tanto, para un mismo TDA vamos a tener diferentes implementaciones.

La notación de Java que utilizaremos para especificar TDA es la notación de interfaces. Para especificar el TDA Ejemplo, notaremos:

```
public interface EjemploTDA {  
}
```

Para especificar una operación de un TDA utilizaremos un prototipo de un método en Java, que incluye el nombre de la operación, los parámetros que recibe con su tipo, y el tipo de dato que retornan. Además, se incluirá un comentario de lo que hace la operación, si no queda claro del nombre y las precondiciones de la misma. Por ejemplo:

```
public interface EjemploTDA {  
    // Descripción de la acción que tiene el método  
    int metodo1(int a, int b);  
}
```

TDA con estructura lineal

Vamos a comenzar a especificar TDA con comportamientos ya conocidos de cursos previos para entender el concepto, para luego avanzar con nuevas estructuras. Durante todo el curso, vamos a suponer que el conjunto de valores que se almacenarán en las diferentes estructuras serán enteros. Esto no es una restricción de los TDA ni tampoco del lenguaje, sino una forma de simplificar la codificación.

2Pila

El primer TDA a especificar será el tipo de dato abstracto que describe el comportamiento de una estructura pila. La pila es una estructura que permite almacenar conjuntos de valores, eliminarlos y recuperarlos, con la particularidad de que el elemento que se recupera o elimina es el último que ingresó. Por lo tanto, para poder definir el comportamiento de la pila vamos a utilizar la siguiente lista de operaciones:

- Apilar: permite agregar un elemento a la pila. Se supone que la pila está inicializada.
- Desapilar: permite eliminar el último elemento agregado a la pila. Se supone como precondición que la pila no esté vacía.
- Tope: permite conocer cuál es el último elemento ingresado a la pila. Se supone que la pila no está vacía.

- PilaVacía: indica si la pila contiene elementos o no. Se supone que la pila está inicializada.
- InicializarPila: permite inicializar la estructura de la pila.

Luego de listar las operaciones del TDA pila vamos a describir el mismo con Java:

```
public interface PilaTDA {
    void InicializarPila();
    // siempre que la pila esté inicializada
    void Apilar(int x);
    // siempre que la pila esté inicializada y no esté vacía
    void Desapilar();
    // siempre que la pila esté inicializada
    boolean PilaVacía();
    // siempre que la pila esté inicializada y no esté vacía
    int Tope();
}
```

A continuación, vamos a realizar un ejemplo en donde utilizaremos la PilaTDA definida anteriormente. En el ejemplo, vamos a pasar los elementos de una pila origen a una pila destino, en donde se perderán los elementos de la pila origen y en la pila destino quedarán esos elementos en orden inverso al del origen (TP1, ejercicio 2.a)

```
public void PasarPila(PilaTDA origen, PilaTDA destino){
    while (!origen.PilaVacía()){
        destino.Apilar(origen.Tope());
        origen.Desapilar();
    }
}
```

Otro ejemplo de utilización de la PilaTDA, es si queremos escribir un método que nos devuelva la suma de los valores que contiene la pila. En éste caso el método va a retornar un valor del tipo int. (TP1, ejercicio 2.e)

```
float sumar(PilaTDA p1) {
    PilaTDA aux=new PilaTF();
    aux.inicializarPila();
    copiar(p1,aux);
    int sum=0;
    while(!aux.pilaVacía()) {
        sum+=aux.tope();
        aux.desapilar();
    }
    return sum;
}
```

Cola

Luego de haber introducido el concepto de TDA, y para ello utilizar la estructura pila, ahora vamos a ver otra estructura que es la cola. Una cola es una estructura que nos permite almacenar valores, recuperar y eliminar el primer valor ingresado, además de que nos permite conocer si la cola tiene o no elementos. Por lo tanto, si analizamos la diferencia en cuanto al comportamiento de una pila y una cola, esa diferencia esta dada en la forma en que recuperan y eliminan los valores en cada estructura.

A continuación, vamos a listar las operaciones, junto a sus comportamientos, que nos permite especificar el TDA de una cola:

- Acolar: permite agregar un elemento a la cola. Se supone que la cola está inicializada.
- Desacolar: permite eliminar el primer elemento agregado a la cola. Se supone como precondition que la cola no esté vacía.
- Primero: permite conocer cuál es el primer elemento ingresado a la cola. Se supone que la cola no está vacía.
- ColaVacía: indica si la cola contiene elementos o no. Se supone que la cola está inicializada.
- InicializarCola: permite inicializar la estructura de la cola.

Es decir, el tipo de dato abstracto ColaTDA nos quedaría de la siguiente manera:

```
public interface ColaTDA { void
    InicializarCola();
    // siempre que la cola esté inicializada
    void Acolar(int x);
    // siempre que la cola esté inicializada y no esté vacía
    void Desacolar();
    // siempre que la cola esté inicializada
    boolean ColaVacía();
    // siempre que la cola esté inicializada y no esté vacía
    int Primero();
}
```

De la misma manera que utilizamos como ejemplo pasar los elementos de una pila a otra, ahora vamos a escribir un método que nos permita pasar los elementos de una cola a otra. En este caso particular, a diferencia de las pilas, por el comportamiento particular de las colas, los elementos en la cola destino quedarán en el mismo orden que en la cola origen. (TP1, ejercicio 4.a)

```
public void pasarCola(ColaTDA origen, ColaTDA destino){
    while (!origen.ColaVacía()){
        destino.Acolar(origen.Primer());
        origen.Desacolar();
    }
}
```

Cola con Prioridad

En las secciones anteriores definimos estructuras en donde los elementos se recuperan y eliminan de acuerdo al orden en que habían ingresado a la estructura, es decir, en el caso de una pila se podía recuperar o eliminar el último elemento ingresado, y en el caso de la cola el primero ingresado. En esta sección vamos a especificar una estructura que a cada elemento cuando ingresa a la misma se le va a asignar una prioridad, y luego los elementos van a ser recuperados o eliminados de la estructura de acuerdo a esa prioridad. A esta estructura la denominaremos cola de prioridad. Este tipo de estructura es utilizado, por ejemplo, en las colas para embarazadas o discapacitados de los supermercados, ó en las colas de espera en las guardias de los hospitales, en dónde en este último caso la prioridad de atención al paciente se le da de acuerdo con el síntoma de problema de salud que haya llegado al hospital.

A continuación vamos a listar las operaciones, junto a sus comportamientos, que nos permite especificar el TDA de una cola con prioridad:

- AcolarPrioridad: permite agregar un elemento a la cola con una cierta prioridad dada. Se supone que la cola está inicializada.
- Desacolar: permite eliminar el elemento con mayor prioridad en la cola (en caso de tener dos elementos con la misma prioridad sale el primero ingresado). Se supone como precondition que la cola no esté vacía.
- Primero: permite conocer cuál es el elemento de mayor prioridad ingresado a la cola (en caso de tener dos elementos con la misma prioridad devuelve el primero ingresado). Se supone que la cola no está vacía.
- ColaVacía: indica si la cola contiene elementos o no. Se supone que la cola está inicializada.
- Prioridad: permite conocer la prioridad del elemento con mayor prioridad de la cola.
- InicializarCola: permite inicializar la estructura de la cola.

Es decir, el tipo de dato abstracto ColaPrioridadTDA nos quedaría de la siguiente manera:

```
public interface ColaPrioridadTDA { void InicializarCola();
    // siempre que la cola esté inicializada
    void AcolarPrioridad(int x, int prioridad);
    // siempre que la cola esté inicializada y no esté vacía
    void Desacolar();
    // siempre que la cola esté inicializada y no esté vacía
    int Primero();
    // siempre que la cola esté inicializada
    boolean ColaVacía();
    // siempre que la cola esté inicializada y no esté vacía int Prioridad();
}
```

Podemos establecer las siguientes conclusiones. Si acolamos todos los elementos con la misma prioridad, la cola con prioridad se comporta como una cola común. Si acolamos

todos los elementos con la prioridad igual a su valor, la cola con prioridad puede ordenar automáticamente los elementos.

Conjunto

El siguiente tipo de dato abstracto que vamos a definir será el conjunto. El conjunto es una estructura que nos permite guardar elementos sin que los mismos se repitan y en donde no se tiene un orden, y además nos permite conocer si un elemento dado se encuentra o pertenece a la estructura. Debido a que la estructura no tiene un orden, cuando recuperamos un dato la estructura nos devuelve uno cualquiera que pertenece a ella, y por otro lado cuando queremos eliminar un valor debemos indicarle cuál es. A partir de esta descripción de comportamiento del conjunto, podemos decir que la lista de operaciones que define el TDA de conjunto es la siguiente:

- Agregar: permite agregar un elemento al conjunto. Se supone que el conjunto está inicializado.
- Sacar: permite eliminar del conjunto un elemento dado. Se supone como precondition que el conjunto no está vacío.
- Elegir: devuelve un elemento cualquiera del conjunto. Se supone como precondition que el conjunto no está vacío.
- Pertenece: permite conocer si un elemento dado se encuentra en el conjunto. Se supone que el conjunto está inicializado.
- ConjuntoVacío: indica si el conjunto contiene elementos o no. Se supone que el conjunto está inicializado.
- InicializarConjunto: permite inicializar la estructura del conjunto.

Es decir, el tipo de dato abstracto ConjuntoTDA nos quedaría de la siguiente manera:

```
public interface ConjuntoTDA {
    void InicializarConjunto();
    // siempre que el conjunto esté inicializado boolean ConjuntoVacío();
    // siempre que el conjunto esté inicializado
    void Agregar(int x);
    // siempre que el conjunto esté inicializado y no esté vacío int Elegir();
    // siempre que el conjunto esté inicializado
    void Sacar(int x);
    // siempre que el conjunto esté inicializado
    boolean Pertenece(int x);
}
```

Como ejemplo de uso del ConjuntoTDA, supongamos que queremos indicar si dos conjuntos son iguales, entendiendo que dos conjuntos son iguales si tienen los mismos elementos.

```
public static boolean sonIguales(ConjuntoTDA a, ConjuntoTDA b) {
    ConjuntoTA aux1= new ConjuntoTA();aux1.inicializarConjunto();
    ConjuntoTA aux2= new ConjuntoTA();aux2.inicializarConjunto();
    copiar(a,aux1);copiar(b,aux2);

    boolean iguales=true;

    int elemento;

    while (!aux1.conjuntoVacio() && !aux2.conjuntoVacio() && iguales ) {
        elemento=aux1.elegir();

        if (!aux2.pertenece(elemento))
            iguales=false;

        else {
            aux1.sacar(elemento);
            aux2.sacar(elemento);
        }
    }

    return (aux1.conjuntoVacio() && aux2.conjuntoVacio());
}
```

Implementación

*****Cuadrado Estrebou, María Fernanda Trutner, Guillermo Hernán. Revisión: J.Tempio.
Programación II. Apuntes de cátedra.***

El objetivo de esta unidad es entender el concepto de implementación y su clara diferencia con la especificación de un TDA. Para eso vamos a ver diferentes implementaciones de los TDA definidos anteriormente, utilizando como estrategia de implementación estructuras de tamaño fijo, es decir, arreglos. Luego de haber realizado las diferentes implementaciones veremos el costo asociado a cada una de las operaciones de cada TDA de acuerdo a la implementación realizada.

Implementación de TDAs

Durante esta sección veremos diferentes implementaciones de PilaTDA, ColaTDA, ColaPrioridadTDA y ConjuntoTDA.

Cuando hablamos de implementación nos estamos refiriendo a que debemos definir los siguientes puntos:

- En dónde se va a guardar la información

- De qué manera se va guardar esa información, es decir, definir el criterio con que se va a guardar y recuperar los datos
- Escribir / implementar cada una de las operaciones del TDA que se esta implementando

En esta sección las estructuras utilizadas para guardar la información van a ser arreglos, lo cual va a tener una restricción que es que tienen tamaño fijo. En secciones posteriores veremos otra forma de guardar la información.

Para implementar un TDA en Java, vamos a escribir una clase, a la que le indicaremos que es la implementación de un TDA con la palabra reservada implements. Luego definiremos la estructura de datos, y por último, codificaremos cada uno de los métodos especificados. Si fuera necesario, se podrán agregar métodos auxiliares, que serán internos a la clase.

```
public class Clase implements interface {  
  
}
```

Implementación de PilaTDA

Como hemos expresado en la unidad anterior, para una misma especificación de TDA pueden existir diferentes implementaciones, y de hecho, existen muchas. Por ese motivo para PilaTDA vamos a ver algunas posibles estrategias.

Estrategia 1:(Tope al final) Implementación en donde se guardan los datos en un arreglo y además se tiene una variable que indica la cantidad de elementos que se tienen guardados en la pila. Cuando agregamos un nuevo elemento a la pila, el mismo se guarda en la posición indicada por la variable que me indica la cantidad de elementos. Cuando se tiene que desapilar un elemento de la pila, solo es necesario decrementar en una unidad la variable que me indica la cantidad de elementos.

Se apila en el siguiente orden: 0,1,2,3,4,5

0	1	2	3	4	5				
0	1	2	3	4	5	6	7	8	9

A continuación, se muestra el código de esta implementación:

```
public class PilaTF implements PilaTDA {  
    int[] a;  
    int indice;  
  
    public void inicializarPila() {
```

```
// TODO Auto-generated method stub

a=new int[100]; // Declara un array de 100 elementos
indice=0; // indice también es contador

}

public void Apilar(int x){
    a[indice] = x;
    indice++;
}

public void Desapilar(){
    indice--;
}

public boolean PilaVacia(){
    return (indice == 0);
}

public int Tope(){
    return (índice-1);
}

}
```

Estrategia 2: Otra variante de implementación es utilizar también un arreglo para almacenar los datos y tener una variable que indica la cantidad de elementos que se tienen guardados en la pila, pero cuando agregamos un nuevo elemento a la pila, en vez de hacerlo en la posición señalada por la variable que me indica la cantidad de elementos, se guarda en la primera posición (es decir, la posición 0 del arreglo). Esto tiene como desventaja que cuando se agrega un elemento se debe hacer un corrimiento del resto de los elementos hacia la derecha, y cuando se desapila un elemento de la pila nuevamente se tiene que hacer un corrimiento de elementos, pero esta vez hacia la izquierda. *Cuando el corrimiento es a izquierda empezamos por la izquierda y cuando el corrimiento es a derecha, comenzamos de la derecha.*

Se puede observar que la variante entre esta implementación y la anterior no esta dada en la estructura utilizada para guardar los elementos sino en el criterio utilizado para almacenar esa información.

A continuación, se muestra el código de esta implementación:

Se apila en el siguiente orden: 0,1,2,3,4,5

5	4	3	2	1	0				
0	1	2	3	4	5	6	7	8	9

```
public class PilaTI implements PilaTDA {
    int[] a;
```

```

int indice;

public void inicializarPila() {
    a = new int[100];
    indice = 0;
}

public void apilar(int x) {
    for(int i = indice - 1; i >= 0; i--) { // Corrimiento a derecha
        a[i+1] = a[i];
    }
    a[0] = x;
    indice++;
}

public void desapilar() {
    for(int i = 0; i < indice; i++) { // Corrimiento a izquierda
        a[i] = a[i+1];
    }
    indice--;
}

public boolean pilaVacia() {
    return (indice == 0);
}

public int tope() {
    return a[0];
}
}

```

Estrategia 3: Podemos mencionar otra estrategia de implementación en donde variamos la estructura utilizada para guardar la información, es decir, también utilizamos un arreglo pero para guardar la cantidad de elementos que tenemos en vez de hacerlo en una variable utilizamos la posición 0 del arreglo.

Implementación de ColaTDA

De la misma manera que para la estructura de Pila vamos a presentar diferentes estrategias de implementación para la estructura Cola. Todas las estrategias de implementación se basarán por el momento en estructuras de arreglos.

Estrategia 1: Cola Primero es el último. Último elemento ingresado a la cola en la posición inicial del arreglo: Implementación en donde se guardan los datos en un arreglo y además se tiene una variable que indica la cantidad de elementos que se tienen guardados en la cola. Cuando agregamos un nuevo elemento a la cola, el mismo se guarda en la posición cero del arreglo, por lo cual se requiere previo a la inserción un corrimiento a derecha de los elementos que se encuentran en la cola. Cuando se tiene que desacolar un elemento de la cola, solo es necesario decrementar en una unidad la variable que me indica la cantidad de elementos. A continuación, se muestra el código de esta implementación:

Se acola en el siguiente orden: 0,1,2,3,4,5

Ejemplo:

5	4	3	2	1	0				
0	1	2	3	4	5	6	7	8	9

```
public class ColaPU implements ColaTDA {
```

```
    int []a;
```

```
    int indice;
```

```
    public void inicializarCola() {
```

```
        a = new int [100];
```

```
        indice = 0;
```

```
    }
```

```
    public void acolar(int x) {
```

```
        for (int i = indice - 1; i >= 0; i--) {// Corrimiento a derecha
```

```
            a[i +1] = a[i];
```

```
        }
```

```
        a[0] = x;
```

```
        indice++;
```

```
    }
```

```
    public void desacolar() {
```

```
        indice--;
```

```
    }
```

```
    public boolean colaVacia() {
```

```

        return (indice == 0);
    }

    public int primero() {
        return a[indice - 1];
    }
}

```

Estrategia 2: (Primero se encuentra al inicio del array) Primer elemento ingresado a la cola en la posición inicial del arreglo: Implementación en donde se guardan los datos en un arreglo y además se tiene una variable que indica la cantidad de elementos que se tienen guardados en la cola. Cuando agregamos un nuevo elemento a la cola, el mismo se guarda en la posición indicada por la variable que guarda la cantidad de elementos que tiene la cola hasta el momento. Cuando se tiene que desacolar un elemento de la cola, se requiere un corrimiento a izquierda en una posición de los elementos que se encuentran en la cola. A continuación, se muestra el código de esta implementación:

Se acola en el siguiente orden: 0,1,2,3,4,5

0	1	2	3	4	5				
0	1	2	3	4	5	6	7	8	9

```

public class ColaPI implements ColaTDA {
    int [] a;
    int indice;

    public void inicializarCola() {
        a = new int[100];
        indice = 0;
    }

    public void acolar(int x) {
        a[indice] = x;
        indice++;
    }

    public void desacolar() {
        for(int i = 0; i < indice-1; i++) {//Corrimiento a izquierda
            a[i] = a[i+1];
        }
    }
}

```

```

        indice--;
    }

    public boolean colaVacia(){
        return (indice == 0);
    }

    public int primero() {
        return a[0];
    }
}

```

Estrategia 3: De la misma manera que para Pilas, podemos mencionar otra estrategia de implementación en donde variamos la estructura utilizada para guardar la información, es decir, también utilizamos un arreglo, pero para guardar la cantidad de elementos que tenemos en vez de hacerlo en una variable utilizamos la posición 0 del arreglo.

Implementación de ColaPrioridadTDA

Dado que el costo en una cola prioridad, va a estar necesariamente en acolar (ya que el elemento debe encontrar la posición en función de su prioridad con respecto al resto de la cola) utilizaremos una variante de la estrategia de ColaPU, que tenía un costo alto en el acolar, pero mantenía bajo costo el desacolar. Dentro de las posibles implementaciones para la cola de prioridad veremos las siguientes:

Estrategia 1: Consideramos como estructuras dos arreglos, en uno de ellos tendremos los elementos y en el otro la prioridad de cada uno de esos elementos. Como veremos se tendrán que mantener ambos arreglos sincronizados en cuanto a que para una posición dada se tendrá en un arreglo el valor del elemento y en el otro la prioridad que le corresponde a ese elemento. Además, como en todos los casos que hemos trabajado con arreglos tendremos una variable en donde llevemos la cantidad de elementos que se tienen. Se puede ver un esquema de esta estrategia en la figura 3.2. El elemento con mayor prioridad estará en la posición anterior a la marcada por índice.

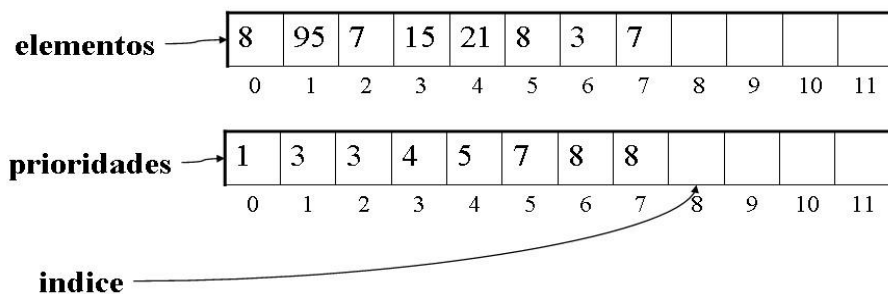


Figura 3.2: Implementación ColaPrioridadTDA con estrategia 1

Supongamos la prioridad tiene el mismo valor que el elemento: Si agrego el 60 con prioridad 60 iría en la posición 6. Pero si agrego 45, con prioridad 45, desplazo el 50 del array a la derecha (posición 6) y acolo el 45 en la posición 5

0	10	20	30	40	50				
0	1	2	3	4	5	6	7	8	9

```
public class ColaPrioridadDA implements ColaPrioridadTDA {
    int [] elementos;
    int [] prioridades;
    int indice;

    public void inicializarCola() {
        elementos = new int[100];
        prioridades = new int[100];
        indice = 0;
    }

    public void acolarPrioridad(int x, int prioridad) {
        int j = indice;
        // Encuentra lugar donde acolar al nuevo elemento desplazando a derecha
        for(;j>0 && prioridades[j-1]>=prioridad; j--) {
            elementos[j] = elementos[j-1]; // Desplazo a derecha
            prioridades[j] = prioridades[j-1]; // Desplazo a derecha
        }
        elementos[j] = x;
        prioridades[j] = prioridad;
        indice++;
    }

    public void desacolar() {
        indice--;
    }

    public boolean colaVacia() {
        return(indice==0);
    }
}
```

```

    }

    public int primero() {
        return elementos[indice-1];
    }

    public int prioridad() {
        return prioridades[indice-1];
    }
}

```

Estrategia 2: Otra opción de implementación de las colas de prioridades es considerar un arreglo cuyos elementos representan una estructura (por ser en Java una clase) que contiene el valor y la prioridad asociada a ese valor. Se puede ver un esquema de esta implementación en la figura 3.3.

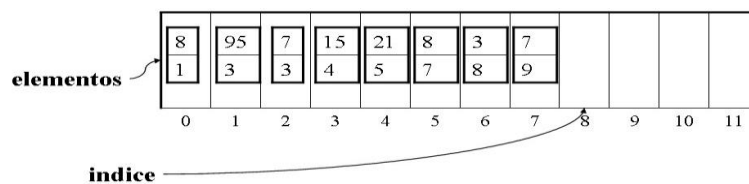


Figura 3.3: Implementación de ColaPrioridadTDA con estrategia 2

A continuación, se muestra el código de esta implementación.

```

public class ColaPrioridadAO implements ColaPrioridadTDA {

    class Elemento{
        int valor;
        int prioridad;
    }

    Elemento[] elementos; // int[] elementos
    int indice;

    public void inicializarCola() {
        indice = 0;
        elementos = new Elemento[100]; // new int[100]
    }
}

```

```

public void acolarPrioridad(int x, int prioridad) {
    int j = indice;
    //desplaza a derecha los elementos de la cola mientras
    //estos tengan mayor o igual prioridad
    for(;j>0 && elementos[j-1].prioridad>=prioridad;j--) {
        elementos[j] = elementos[j-1];
    }
    elementos[j] = new Elemento();
    elementos[j].valor = x;
    elementos[j].prioridad = prioridad;
    indice++;
}

public void desacolar() {
    elementos[indice-1] = null;
    indice--;
}

public boolean colaVacia() {
    return(indice == 0);
}

public int primero() {
    return elementos[indice-1].valor;
}

public int prioridad() {
    return elementos[indice-1].prioridad;
}
}

```

Implementación de ConjuntoTDA

A continuación, vamos a llevar a cabo la implementación del TDA de conjuntos, a partir de la utilización de un arreglo. En el caso del conjunto debido a que los elementos no tienen un orden, cuando se elimina un elemento se puede colocar el último elemento que se encuentra en el arreglo en la posición del elemento a eliminar, y de esta manera evitar el desplazamiento de todos los elementos. Para implementar el método Elegir, dado que la especificación no indica cuál es el elemento para recuperar, vamos a recuperar el elemento que está en la primera posición del arreglo.

0	10	20	30	40	50				
0	1	2	3	4	5	6	7	8	9

```

public class ConjuntoTA implements ConjuntoTDA {
    int[] a;
    int cant;

    public void inicializarConjunto() {
        a = new int[100];
        cant = 0;
    }

    public void agregar(int x) {
        if(!this.pertenece(x)) { //this reemplaza por el nombre del objeto
            a[cant]=x;
            cant++;
        }
    }

    public void sacar(int x) {
        int i = 0; // Busco el elemento
        while(i<cant && a[i]!=x) {
            i++;
        }
        if(i<cant) {
            // Si lo encuentra lo reemplazo por el ultimo y decremento el cant

```

```

        a[i] = a[cant-1];
        cant--;
    }
}

public boolean conjuntoVacio() {
    return (cant==0);
}

public int elegir() {
    Random rand=new Random();
    int x;
    x= rand.nextInt(cant);
    return a[x];
}

public boolean pertenece(int x) {
    int i = 0;
    while (i<cant && a[i]!=x) {
        i++;
    }
    return (i<cant);
}
}

```

Una implementación particular de conjuntos es la que denominamos conjunto de universo acotado, en donde tenemos un arreglo cuyas posiciones representan la presencia o ausencia de un valor en el conjunto. Es decir, si consideramos un conjunto de 15 elementos (del 0 al 14), tendremos un arreglo de 15 posiciones, en donde tendremos por ejemplo un 1 en la posición 5 si es que el elemento 5 pertenece al conjunto, y 0 en caso contrario. Esta implementación tiene limitaciones en cuanto a los valores que puede tomar el conjunto, pero puede ser útil para aquellos casos en donde esta limitación no sea un problema ya que tenemos una manera más eficiente de agregar o eliminar un valor del arreglo, así como de conocer si un cierto valor pertenece al mismo. Todas las operaciones sobre este tipo de implementación para el conjunto son de costos constantes.

0	0	0	0	0	0	0	0	0	0
0	1	2	3	4	5	6	7	8	9

Bibliografía

- [AA98] J. Hopcroft Y J. Ullman A. Aho. Estructuras de Datos y Algoritmos. Addison Wesley, Ubicación en biblioteca UADE: 681.3.01/A292, 1998.
- [Wei04] Mark Allen Weiss. Estructuras de datos en JAVA. Addison Wesley, Ubicación en biblioteca UADE: 004.43 WEI est [1a] 2004, 2004.
- [yJC06] J. Lewis y J. Chase. Estructuras de Datos con Java. Diseño de Estructuras y Algoritmos. Addison Wesley Iberoamericana, 2006.
- [yRT02] M. Goodrich y R. Tamassia. Estructura de Datos y Algoritmos en Java. Editorial Continental, 2002.